🔵 Core Fundamentals

- What is Kafka & why use it (vs RabbitMQ, ActiveMQ)
- Kafka Architecture overview
- Topics, Partitions, Offsets
- Brokers & Kafka Cluster
- Zookeeper vs KRaft (Zookeeper-free Kafka)

🔵 Producers

- Producer API basics
- Acknowledgment modes (acks = 0, 1, all)
- Idempotent Producer
- Compression (gzip, snappy, lz4)
- Batching & Linger.ms
- Partitioner strategies (Round-robin, Key-based, Custom)
- Retries & Delivery guarantees

🔵 Consumers

- Consumer API basics
- Consumer Groups & Load balancing
- Partition rebalancing strategies (Eager, Cooperative)
- Offset management (Auto vs Manual commit)
- At-most-once, At-least-once, Exactly-once semantics
- Lag monitoring
- poll() loop internals

🔵 Kafka with Java (Spring Boot)

- Spring Kafka (@KafkaListener, KafkaTemplate)
- Producer & Consumer configuration in Spring
- Error handling & Dead Letter Topics (DLT)
- Retry mechanisms
- Serialization / Deserialization (String, JSON, Avro)

- Transactions in Kafka with Spring

🔵 **Kafka Internals**

- Log segments & Retention policies (time, size)
- Replication factor & ISR (In-Sync Replicas)
- Leader election
- Compaction (Log compaction)
- Page cache & Zero-copy
- Exactly-once semantics (EOS) internals

🔵 **Kafka Streams**

- Kafka Streams vs Kafka Consumer API
- KStream vs KTable vs GlobalKTable
- Stateless operations (map, filter, flatMap)
- Stateful operations (aggregations, joins, windowing)
- State stores (in-memory, RocksDB)
- Topology & Processing guarantees

🔵 **Schema Registry & Avro**

- Why Schema Registry?
- Avro schema definition
- Schema evolution & compatibility modes (BACKWARD, FORWARD, FULL)
- Integrating Confluent Schema Registry with Java

🔵 **Reliability & Guarantees**

- Exactly-once vs At-least-once — trade-offs
- Transactions API (beginTransaction, commitTransaction)
- Idempotent consumers
- Handling duplicate messages

🔵 **Performance & Tuning**

- Key producer configs (batch.size, linger.ms, buffer.memory)
- Key consumer configs (fetch.min.bytes, max.poll.records)
- Partition count decisions
- Throughput vs Latency trade-offs
- Consumer lag & scaling strategies

🔵 **Operations & Monitoring**

- Kafka CLI commands (topics, consumers, producers)
- Monitoring tools — Kafka UI, Confluent Control Center, Grafana + Prometheus
- Common issues — consumer lag, rebalancing storms, broker failures
- Kafka Connect basics (source & sink connectors)

🔵 **Security**

- Authentication — SSL, SASL (PLAIN, SCRAM, Kerberos)
- Authorization — ACLs
- Encryption in transit vs at rest

🔵 **Scenario-based Questions (Must Prep)**

- How do you ensure no message loss?
- How do you handle duplicate processing?
- How would you design a Kafka-based order processing system?
- What happens when a consumer is down for a long time?
- How do you scale consumers?
- How do you handle poison pill messages?

🔵 Topic 1: What is Kafka & Why Use It?

What is Kafka?

Apache Kafka is a **distributed event streaming platform**. In simple terms, it is a system that allows applications to **send, store, and receive messages/events** in real-time at massive scale.

Think of it like a **postal service**:

- Someone **sends** a letter (Producer)
- The **post office stores** it (Kafka Broker)
- Someone **receives** it (Consumer)

Real-world Analogy

Imagine a **food delivery app** like Zomato:

- When you place an order → an event is generated
- That event needs to reach → the restaurant, delivery agent, payment system, notification service **simultaneously**
- Kafka acts as the **backbone** that carries all these events reliably and in real-time

Why Kafka? (Problems it solves)

Without Kafka, services talk to each other **directly**, which causes:

- Tight coupling between services
- If one service is down, data is lost
- Hard to scale

With Kafka:

🔷 **1. High Performance**

- Handles **millions of messages per second**
- Uses **sequential disk writes** → very fast

🔷 **2. Scalability**

- Easily scale by adding more brokers
- Topics can be partitioned across machines

🔷 **3. Fault Tolerance**

- Data is **replicated across brokers**
- No single point of failure

🔷 **4. Durability**

- Messages are stored on disk (not just memory)
- Can **replay events anytime**

Kafka vs RabbitMQ vs ActiveMQ

🔷 **Event Streaming (Kafka) → Log**

Apache Kafka

- Think of it like a **history book / log file**
- Events are **appended continuously**
- Data is **not deleted after reading**
- Consumers can **re-read (replay)** anytime

👉 Example:
User clicks → stored as event → analytics service can read it now OR later again

✓ Key idea: **"Store everything, read anytime"**

🔷 **Message Broker (Queue) → Queue**

RabbitMQ, **Apache ActiveMQ**

- Think of it like a **task queue**
- Message is **consumed once and removed**
- Focus is on **delivery, not history**

👉 Example:
Order placed → sent to queue → worker processes → message gone

✓ Key idea: **"Process once and discard"**

Where is Kafka Used in Real World?

- **LinkedIn** — Activity tracking (who viewed your profile)
- **Uber** — Real-time trip events
- **Netflix** — Real-time recommendations
- **Banks** — Fraud detection pipelines
- **E-commerce** — Order processing, inventory updates

🔵 Topic 2: Kafka Architecture

The Big Picture

Kafka's architecture has a few key components that work together. Let's understand each one clearly.

[Producer] ——→ [Kafka Cluster (Brokers)] ——→ [Consumer]

|

[Zookeeper / KRaft]

🧩 Component 1: Producer

- Any application that **sends (publishes) messages** to Kafka
- Example: Your order service sends an "order placed" event to Kafka

---

🧩 Component 2: Consumer

- Any application that **reads (subscribes to) messages** from Kafka
- Example: Your notification service reads the "order placed" event and sends an SMS

🧩 Component 3: Broker

- A **Kafka server** that stores messages
- A single Kafka installation can have **multiple brokers** forming a **cluster**
- Each broker has a unique ID (e.g., Broker 1, Broker 2, Broker 3)
- Brokers handle all read and write requests

Analogy: Think of a broker as a **warehouse** that stores parcels (messages) until they are picked up.

🧩 Component 4: Kafka Cluster

- A group of **multiple brokers** working together
- Provides **fault tolerance** — if one broker goes down, others continue working
- Provides **scalability** — you can add more brokers as load increases

Kafka Cluster

├── Broker 1

├── Broker 2

└── Broker 3

🧩 Component 5: Topic

- A **category or feed name** to which messages are published
- Like a **folder** where related messages are stored
- Example topics: order-events, payment-events, user-signups
- Producers write to a topic, Consumers read from a topic

Analogy: Think of a topic as a **YouTube channel** — producers upload videos (messages), consumers watch (read) them.

🧩 Component 6: Partition

- Each topic is split into **multiple partitions**
- Partitions allow Kafka to **scale horizontally** and process messages in **parallel**
- Each partition is an **ordered, immutable sequence** of messages
- Messages within a partition have a unique ID called an **Offset**

Topic: order-events

├── Partition 0: [msg1, msg2, msg3 ...]

├── Partition 1: [msg4, msg5, msg6 ...]

└── Partition 2: [msg7, msg8, msg9 ...]

Analogy: Think of a topic as a **highway** and partitions as **lanes** — more lanes = more traffic can flow simultaneously.

🧩 Component 7: Offset

- A **unique sequential number** assigned to each message within a partition
- Offsets are **per partition** (not global)
- Consumers use offsets to track **which messages they have already read**

Partition 0:

Offset → 0 1 2 3 4

Message → msg1 msg2 msg3 msg4 msg5

⚠️ Order is **guaranteed within a partition**, but NOT across partitions.

🧩 Component 8: Replication

- Each partition is **copied (replicated)** across multiple brokers for fault tolerance
- One broker holds the **Leader** replica (handles all reads/writes)
- Other brokers hold **Follower** replicas (stay in sync with leader)
- If the leader goes down, a follower is **automatically promoted** as the new leader

Partition 0:

├── Broker 1 → Leader ✅ (handles reads/writes)

├── Broker 2 → Follower (in sync)

└── Broker 3 → Follower (in sync)

🧩 Component 9: Zookeeper / KRaft

- **Zookeeper** was traditionally used to manage Kafka's metadata:
  - Which broker is the leader for which partition?
  - Which brokers are alive?
  - Cluster configuration
- **KRaft** (Kafka Raft) is the **new replacement for Zookeeper** (from Kafka 3.x onwards)
  - Kafka now manages its own metadata internally
  - Simpler architecture, better performance

🏗️ Putting It All Together

Here's a complete flow of how Kafka works:

1. Producer sends a message to Topic "order-events"

↓

2. Kafka decides which Partition to store it in

↓

3. Message is stored in the Leader Broker for that Partition

↓

4. Message is replicated to Follower Brokers

↓

5. Consumer reads the message from the Partition using Offset

↓

6. Consumer commits the Offset (marks message as read)

🔵 Topic 3: Topics, Partitions & Offsets

📌 Part 1: Topics

What is a Topic?

A topic is a **named stream of messages** in Kafka. It is the fundamental way to **organize and categorize** messages.

- Producers **write to** topics
- Consumers **read from** topics
- A topic can have **multiple producers** and **multiple consumers**
- Topics are identified by a **unique name** (e.g., order-events, payment-events)

Topic Properties

- Topics are **append-only** — new messages are always added at the end
- Messages in a topic are **immutable** — once written, they cannot be changed
- Messages are **retained** for a configurable period (default 7 days), even after consumption
- Unlike RabbitMQ, messages are **NOT deleted** after a consumer reads them

---

Real World Example

Imagine an e-commerce platform with these topics:

Topics:

├── order-placed-events

├── payment-processed-events

├── shipment-events

├── notification-events

└── inventory-events

Each service produces to and consumes from relevant topics independently.

📌 Part 2: Partitions

What is a Partition?

A partition is a **sub-division of a topic**. Each topic is broken into one or more partitions.

Topic: order-placed-events (3 partitions)

├── Partition 0

├── Partition 1

└── Partition 2

Why Partitions?

Partitions solve two major problems:

1. Scalability

- A single broker cannot handle millions of messages per second
- By splitting a topic into partitions, the load is **distributed across multiple brokers**
- More partitions = more parallelism = higher throughput

2. Parallel Consumption

- Multiple consumers can read from **different partitions simultaneously**
- Without partitions, only one consumer could read at a time

How are Messages Distributed Across Partitions?

When a producer sends a message, Kafka decides which partition to send it to based on:

Case 1: Message has a Key

Kafka applies: partition = hash(key) % number_of_partitions

Example:

Key = "order-123" → hash = 456789 → 456789 % 3 = 0 → Partition 0

Key = "order-456" → hash = 789012 → 789012 % 3 = 1 → Partition 1

Key = "order-123" → always goes to Partition 0 ✅ (ordering guaranteed)

- Same key **always goes to the same partition**
- This guarantees **ordering for related messages**

Case 2: No Key (null)

Messages are distributed in Round Robin:

Message 1 → Partition 0

Message 2 → Partition 1

Message 3 → Partition 2

Message 4 → Partition 0

...

- Load is evenly balanced
- But ordering across messages is **NOT guaranteed**

Important Rule About Ordering

✅ Order is guaranteed **within a partition**
❌ Order is NOT guaranteed **across partitions**

Partition 0: [order-1-placed → order-1-payment → order-1-shipped] ✅ In order

Partition 1: [order-2-placed → order-2-payment → order-2-shipped] ✅ In order

But across Partition 0 and Partition 1 → ❌ No order guarantee

So if ordering matters (e.g., all events for the same order), always use a **key** so related messages land in the same partition.

Partitions & Brokers

Partitions are distributed across brokers in a cluster:

Kafka Cluster (3 Brokers, 1 Topic, 3 Partitions, Replication Factor = 2)

Broker 1: Partition 0 (Leader), Partition 2 (Follower)

Broker 2: Partition 1 (Leader), Partition 0 (Follower)

Broker 3: Partition 2 (Leader), Partition 1 (Follower)

- Each partition has **one Leader** and **one or more Followers**
- All reads and writes go to the **Leader**
- Followers just **replicate** data from the Leader

How Many Partitions Should You Choose?

This is a common interview question! Here's the thinking:

Factor	Guidance
Higher throughput needed	Increase partitions
More consumer parallelism needed	Increase partitions
Too many partitions	Increases overhead on broker & Zookeeper
Rule of thumb	Start with 3-6, scale based on load
Max consumers per topic	= Number of partitions

⚠️ You can **increase** partitions later, but you **cannot decrease** them.

📌 **Part 3: Offsets**

What is an Offset?

An offset is a **unique, sequential integer** assigned to every message within a partition. It represents the **position** of a message in a partition.

Partition 0:

```
┌──────────────────────────────────────────────┐
| Offset:  0     1     2     3     4     |
| Message: msg1   msg2   msg3   msg4   msg5   |
└──────────────────────────────────────────────┘
```

Partition 1:

```
┌──────────────────────────────────────────────┐
| Offset:  0     1     2     3          |
| Message: msg6   msg7   msg8   msg9        |
└──────────────────────────────────────────────┘
```

Important:

- Offsets start from **0** in each partition
- Offsets are **per partition** — Partition 0 has its own offset 0, Partition 1 has its own offset 0
- Offsets are **immutable** — once assigned, they never change
- Offsets **always increase** — never reset (unless topic is deleted)

How Consumers Use Offsets

Consumers use offsets to track **where they left off** reading.

Consumer reading Partition 0:

Step 1: Read offset 0 (msg1) → process it

Step 2: Read offset 1 (msg2) → process it

Step 3: Read offset 2 (msg3) → process it

Step 4: Commit offset 3 (tells Kafka: "I've read up to offset 2, next give me offset 3")

This is called **offset commit** — it allows consumers to:

- Resume from where they left off after a restart
- Not reprocess already processed messages

Where are Offsets Stored?

Offsets are stored in a special Kafka topic called:

__consumer_offsets

This is an **internal Kafka topic** that maintains committed offsets for all consumer groups.

Types of Offset Reset

When a consumer joins for the first time or loses its offset, Kafka needs to know where to start:

Config	Behavior
earliest	Start reading from the very first message (offset 0)
Latest	Start reading only new messages (ignore old ones)
None	Throw an error if no offset found

🔷 When does earliest / latest actually apply?

1️⃣ **New consumer group (no offset exists)**

New group → No stored offset → Apply earliest/latest

2️⃣ **Offset expired (due to retention)**

Consumer down too long → Offset deleted → Apply earliest/latest

3️⃣ **Offset is invalid (log deleted)**

Offset points to deleted data → Apply earliest/latest

java

// In Spring Boot / Java

props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");

Auto vs Manual Offset Commit

Auto Commit (default)

java

props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, true);

props.put(ConsumerConfig.AUTO_COMMIT_INTERVAL_MS_CONFIG, 5000); // every 5 seconds

- Kafka automatically commits offsets at regular intervals
- Risk: If your app crashes before commit, messages may be **reprocessed**

Manual Commit

java

props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, false);

// After processing:

consumer.commitSync(); // Synchronous - waits for confirmation

consumer.commitAsync(); // Asynchronous - doesn't wait

- You control exactly when the offset is committed
- More reliable for critical applications

🔑 **Key Takeaways**

Concept	One-line Summary
Topic	Named category for messages
Partition	Sub-division of topic for parallelism & scale
Key-based routing	Same key always goes to same partition
Round-robin	Messages without key distributed evenly
Offset	Unique position of a message in a partition
Offset commit	Consumer tracking how far it has read

Concept	One-line Summary
__consumer_offsets	Internal Kafka topic storing committed offsets
earliest / latest	Where to start reading when offset is unknown

---

🎯 **Common Interview Questions on This Topic**

1. **How does Kafka guarantee message ordering?**
   → Only within a partition. Use message keys to ensure related messages go to the same partition.
2. **What happens if a consumer restarts?**
   → It resumes from the last committed offset.
3. **Can you decrease the number of partitions?**
   → No, you can only increase partitions in Kafka.
4. **What is the risk of auto-commit?**
   → Messages may be reprocessed if the app crashes between processing and commit.

🔵 Topic 4: Brokers & Kafka Cluster

👉 *Kafka allows a broker to be leader for multiple partitions; upon failure, a new leader is elected from the ISR, even if that broker is already leading other partitions.*

📌 **Part 1: What is a Kafka Broker?**

A Kafka Broker is simply a **Kafka server** — a machine (or process) that:

- **Stores** messages (in partitions)
- **Receives** messages from Producers
- **Serves** messages to Consumers
- **Replicates** data to other brokers

Each broker is identified by a **unique integer ID** (e.g., Broker 1, Broker 2, Broker 3)

What does a Broker Store?

A broker stores **partition data** on disk in the form of **log files**.

Broker 1 (disk storage):

├── /kafka-logs/order-events-0/ ← Partition 0 of topic order-events

├── /kafka-logs/order-events-2/ ← Partition 2 of topic order-events

├── /kafka-logs/payment-events-1/ ← Partition 1 of topic payment-events

└── ...

Each partition folder contains **segment files** (chunks of messages stored on disk).

Broker is Stateless (kind of)

- Brokers do **NOT track** which consumer has read which message
- That responsibility belongs to the **consumer itself** (via offsets)
- This is why Kafka is extremely **scalable** — brokers just store and serve data

---

📌 Part 2: Kafka Cluster

A **Kafka Cluster** is a group of multiple brokers working together.

Kafka Cluster

├── Broker 1 (ID: 1) → stores some partitions

├── Broker 2 (ID: 2) → stores some partitions

└── Broker 3 (ID: 3) → stores some partitions

Why a Cluster?

Benefit	Explanation
Fault Tolerance	If one broker dies, others continue serving
Scalability	Add more brokers to handle more load
High Availability	Data is replicated across brokers
Load Distribution	Partitions spread across brokers evenly

📌 Part 3: Bootstrap Servers

When a Producer or Consumer connects to Kafka, it doesn't need to know **all** brokers — it just needs to know **one or a few** brokers called **Bootstrap Servers**.

java

// Java configuration

props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "broker1:9092,broker2:9092");

How it works:

1. Producer connects to broker1:9092 (bootstrap server)

↓

2. Broker1 returns metadata about ALL brokers and partitions

↓

3. Producer now knows the full cluster and connects directly

to the correct broker/partition for writing

Even if you list only one bootstrap server, Kafka discovers the rest automatically. But listing 2-3 is recommended for fault tolerance.

📌 Part 4: Replication

Replication is the process of **copying partition data** across multiple brokers to prevent data loss.

Replication Factor

- Defines **how many copies** of each partition exist
- Set at the **topic level**
- Replication Factor = 3 means 3 copies of every partition (1 leader + 2 followers)

Topic: order-events

Replication Factor: 3

Partitions: 3

Partition 0 → Leader on Broker1, Follower on Broker2, Follower on Broker3

Partition 1 → Leader on Broker2, Follower on Broker3, Follower on Broker1

Partition 2 → Leader on Broker3, Follower on Broker1, Follower on Broker2

Rule of Thumb

Replication Factor should **never exceed** the number of brokers.
For production: **Replication Factor = 3**, **Brokers >= 3**

📌 Part 5: Leader & Follower

For every partition, one broker is the **Leader** and the rest are **Followers**.

Leader

- Handles **all reads and writes** for that partition
- Producers always write to the Leader
- Consumers always read from the Leader (by default)

Follower

- **Passively replicates** data from the Leader
- Does NOT serve reads or writes normally
- Acts as a **backup** — ready to take over if leader fails

Partition 0:

```
┌──────────────────────────────────────┐
| Broker 1 → LEADER ✅ R/W      |
| Broker 2 → Follower (syncing...) |
| Broker 3 → Follower (syncing...) |
└──────────────────────────────────────┘
```

📌 Part 6: ISR (In-Sync Replicas)

ISR stands for **In-Sync Replicas** — the set of replicas that are **fully caught up** with the leader.

Partition 0:

Leader: Broker 1 (latest offset = 100)

Broker 2: synced up to offset 100 ✅ → IN ISR

Broker 3: synced up to offset 95 ❌ → OUT of ISR (lagging)

ISR = [Broker1, Broker2]

Why ISR matters?

- When a producer sends with acks=all, Kafka waits for **all ISR replicas** to acknowledge
- Only brokers in ISR are **eligible to become the new leader**

What causes a replica to fall out of ISR?

- Broker is slow or overloaded
- Network issues
- Broker is down

📌 Part 7: Leader Election

What happens when a Leader broker goes down?

Normal State:

Partition 0 → Leader: Broker1, Followers: Broker2, Broker3

Broker1 goes DOWN 💥

Election happens:

→ Kafka picks a new leader from ISR

→ Broker2 becomes new Leader ✅

→ Broker3 becomes Follower of Broker2

Producers and Consumers automatically redirect to Broker2

How is the new leader chosen?

- Kafka (via Zookeeper or KRaft) monitors broker health
- When a leader fails, the **Controller Broker** triggers leader election
- The new leader is chosen from the **ISR list**
- If ISR is empty → Kafka can optionally elect an **out-of-sync replica** (risky — may cause data loss)

📌 Part 8: Controller Broker

Among all brokers in a cluster, one special broker is elected as the **Controller**.

Responsibilities of Controller:

- Monitors other brokers for failures (via Zookeeper/KRaft)
- Triggers **leader election** when a broker goes down
- Manages **partition assignments** when new brokers join
- Only **one controller** exists at a time in the cluster

Kafka Cluster:

├── Broker 1 → CONTROLLER 👑 (manages cluster metadata)

├── Broker 2 → Regular Broker

└── Broker 3 → Regular Broker

If the Controller itself goes down → a new Controller is elected automatically.

📌 Part 9: What Happens When a Broker Goes Down?

Let's walk through the full scenario:

Setup:

- 3 Brokers (1, 2, 3)

- Topic: order-events, 3 partitions, RF=3

- Broker 1 is the Controller

- Partition 0 Leader: Broker 2

Broker 2 goes DOWN 💥

Step 1: Broker 2 stops sending heartbeats to Zookeeper/KRaft

Step 2: Controller (Broker 1) detects Broker 2 is dead

Step 3: Controller looks at ISR for Partition 0 → [Broker1, Broker3]

Step 4: Controller elects Broker 3 as new Leader for Partition 0

Step 5: Controller updates cluster metadata

Step 6: Producers & Consumers get updated metadata

Step 7: Producers now write to Broker 3 for Partition 0

Step 8: Everything continues normally ✅

When Broker 2 comes back:

Step 9: Broker 2 rejoins as a Follower

Step 10: Broker 2 syncs data from current Leader (Broker 3)

Step 11: Once fully synced, Broker 2 rejoins ISR

📌 Part 10: Preferred Leader Election

Over time, after broker restarts and failures, partition leaders may become **unevenly distributed**:

After some failures:

Broker 1 → Leader for 8 partitions 😰 (overloaded)

Broker 2 → Leader for 1 partition

Broker 3 → Leader for 1 partition

Kafka has a concept of **Preferred Leader** — the originally assigned leader for a partition.

Kafka can automatically rebalance leaders back to their preferred brokers:

bash

Enable auto leader rebalance

auto.leader.rebalance.enable=true

🔑 Key Takeaways

Concept	One-line Summary
Broker	Kafka server that stores and serves messages
Cluster	Group of brokers for scale and fault tolerance
Bootstrap Server	Entry point for clients to discover the cluster
Replication Factor	Number of copies of each partition
Leader	Broker that handles all reads/writes for a partition
Follower	Broker that replicates data from leader
ISR	Set of replicas fully in sync with leader
Controller	Special broker managing cluster metadata & elections
Leader Election	Process of picking a new leader when current one fails

---

🎯 Common Interview Questions

1. **What is ISR and why is it important?**
   → ISR contains replicas fully synced with leader. Only ISR members can become leaders, ensuring no data loss.
2. **What happens if all ISR replicas go down?**
   → Kafka can elect an out-of-sync replica (unclean leader election) — risky, may cause data loss. Controlled by unclean.leader.election.enable=false.
3. **What is the role of the Controller broker?**
   → Manages leader elections, monitors broker health, handles partition assignments.
4. **What is the recommended replication factor for production?**
   → 3 — provides fault tolerance even if one broker goes down.
5. **Can consumers read from follower replicas?**
   → By default no. But from Kafka 2.4+, consumers can read from the **nearest replica** for lower latency (rack-aware consumers).

🔵 Topic 5: Zookeeper vs KRaft

📌 Part 1: What is Zookeeper?

Apache Zookeeper is a **distributed coordination service** — a separate system that Kafka used to rely on for managing cluster metadata and coordination.

Think of Zookeeper as the **manager of the Kafka office**:

- Keeps track of which brokers are alive
- Knows who the leader is for each partition
- Stores all cluster configuration
- Notifies Kafka when something changes

Traditional Kafka Architecture:

[Producer] → [Kafka Cluster] → [Consumer]

↕

[Zookeeper Cluster]

(separate system managing metadata)

📌 Part 2: What did Zookeeper do for Kafka?

Responsibility	Details
Broker Registration	Each broker registers itself with Zookeeper on startup
Leader Election	Zookeeper helped elect partition leaders & controller
Cluster Metadata	Stored topic configs, partition assignments, ACLs
Health Monitoring	Tracked which brokers are alive via heartbeats
Consumer Group Offsets	(older versions) Stored consumer offsets

---

📌 **Part 3: Problems with Zookeeper**

Over time, the Kafka community realized Zookeeper was causing several problems:

Problem 1: Separate System to Manage

Without Zookeeper: Deploy & manage only Kafka ✅

With Zookeeper: Deploy & manage Kafka + Zookeeper ❌

- Two separate systems to install, configure, monitor, and scale
- Operational complexity increases significantly

Problem 2: Scalability Bottleneck

Zookeeper can handle ~1 million partitions

Modern Kafka clusters need millions+ of partitions

- Zookeeper becomes a bottleneck at very large scale
- All metadata operations go through Zookeeper — single point of pressure

Problem 3: Metadata Lag

Broker crashes

↓

Zookeeper detects it (slight delay)

↓

Notifies Controller broker (another delay)

↓

Controller triggers leader election (another delay)

↓

New leader elected

- Multiple hops = **slower failover**

- In large clusters this could take **30-60 seconds**

Problem 4: Split Brain Risk

Network partition occurs:

Zookeeper cluster splits into two halves

Each half thinks it's the valid cluster

→ Two controllers exist simultaneously 💥

→ Data inconsistency risk

Problem 5: Duplicated Metadata

- Kafka brokers cached Zookeeper metadata locally
- On startup, brokers had to reload all metadata from Zookeeper
- In large clusters with thousands of partitions → **very slow startup**

📌 **Part 4: What is KRaft?**

KRaft (Kafka Raft) is Kafka's **built-in metadata management system** that completely **replaces Zookeeper**.

- Introduced as experimental in **Kafka 2.8 (2021)**
- Production ready in **Kafka 3.3 (2022)**
- Zookeeper **fully removed** in **Kafka 4.0 (2024)**

The Core Idea

Instead of relying on an external system (Zookeeper), Kafka now manages its own metadata using the **Raft consensus algorithm** internally.

New Kafka Architecture (KRaft):

[Producer] → [Kafka Cluster] → [Consumer]

↕

[KRaft (built into Kafka)]

(no separate Zookeeper needed)

📌 **Part 5: What is the Raft Algorithm?**

Raft is a **consensus algorithm** — a way for a group of servers to **agree on a single value or state** even if some servers fail.

Simple analogy:

Imagine 5 people voting on a decision. As long as majority (3+) agree, the decision is final — even if 2 people are unreachable.

In KRaft:

- A group of brokers act as **voters** (typically 3 or 5)
- They elect a **leader** among themselves
- The leader manages all metadata changes
- Changes are only committed when **majority of voters acknowledge**

KRaft Quorum (3 voters):

├── Broker 1 → KRaft Leader 👑 (handles metadata writes)

├── Broker 2 → KRaft Voter (replicates metadata)

└── Broker 3 → KRaft Voter (replicates metadata)

📌 **Part 6: KRaft Architecture**

In KRaft mode, brokers can have two roles:

Role 1: Controller

- Participates in KRaft **metadata quorum**
- Manages leader elections, topic configs, partition assignments
- Does NOT handle producer/consumer traffic
- Recommended: **3 or 5 dedicated controller nodes** in production

Role 2: Broker

- Handles producer and consumer traffic
- Stores actual message data
- Fetches metadata from KRaft controllers

Role 3: Combined (for small setups)

- A single node acts as both Controller and Broker
- Fine for development, not recommended for production

Production KRaft Setup:

Controllers (metadata quorum):

├── Controller 1 (KRaft Leader)

├── Controller 2 (KRaft Voter)

└── Controller 3 (KRaft Voter)

Brokers (data plane):

├── Broker 1

├── Broker 2

└── Broker 3

📌 Part 7: How KRaft Solves Zookeeper's Problems

Problem	Zookeeper	KRaft
Separate system	Yes — extra ops burden	No — built into Kafka
Scalability	~1M partitions max	Millions+ partitions
Failover speed	30-60 seconds	Few seconds
Split brain risk	Possible	Eliminated via Raft
Startup time	Slow (reload from ZK)	Fast (metadata local)
Operational complexity	High	Low

📌 Part 8: Metadata Storage in KRaft

In KRaft, all metadata is stored in a special internal Kafka topic:

__cluster_metadata

This is a **single-partition, replicated topic** that stores:

- Topic and partition configurations
- Broker registrations
- Leader information
- ACLs and security configs

Traditional (Zookeeper):

Metadata stored in Zookeeper znodes (tree structure)

→ Brokers cache it locally

→ Can get out of sync

KRaft:

Metadata stored in __cluster_metadata topic

→ All brokers subscribe to it

→ Always consistent ✅

📌 **Part 9: KRaft Leader Election Flow**

Normal State:

KRaft Leader = Controller 1

Controller 1 goes DOWN 💥

Step 1: Controller 1 stops sending heartbeats

Step 2: Controller 2 & 3 detect leader is gone (election timeout)

Step 3: Controller 2 starts an election — sends vote request

Step 4: Controller 3 votes for Controller 2

Step 5: Controller 2 wins majority (2/3 votes) → becomes new Leader ✅

Step 6: Controller 2 notifies all brokers

Step 7: Brokers update their metadata — operations resume

Total time: milliseconds to a few seconds ⚡

📌 **Part 10: Migrating from Zookeeper to KRaft**

For teams still on older Kafka versions, migration path is:

Step 1: Upgrade to Kafka 3.x

Step 2: Run in "bridge mode" (both ZK and KRaft active)

Step 3: Migrate metadata from Zookeeper to KRaft

Step 4: Disable Zookeeper completely

Step 5: Upgrade to Kafka 4.x (ZK fully removed)

📌 **Part 11: KRaft Configuration Example**

properties

```
# kraft/server.properties

# Set the role of this node
process.roles=broker,controller   # combined mode (dev only)
# process.roles=controller        # dedicated controller
# process.roles=broker            # dedicated broker

# Node ID (unique per node)
node.id=1

# KRaft controller quorum voters
# format: id@host:port
controller.quorum.voters=1@controller1:9093,2@controller2:9093,3@controller3:9093

# Listeners
listeners=PLAINTEXT://:9092,CONTROLLER://:9093
controller.listener.names=CONTROLLER
```

🔑 **Key Takeaways**

Concept	One-line Summary
Zookeeper	External system that managed Kafka metadata
KRaft	Built-in replacement for Zookeeper using Raft algorithm
Raft	Consensus algorithm ensuring majority agreement
KRaft Controller	Manages cluster metadata using Raft quorum
__cluster_metadata	Internal topic storing all cluster metadata in KRaft
Combined mode	Single node acts as both broker and controller (dev only)
Kafka 4.0	Zookeeper completely removed, KRaft is the only option

---

🎯 **Common Interview Questions**

1. **Why was Zookeeper removed from Kafka?**
   → Operational complexity, scalability bottleneck, slow failover, and split-brain risks led to its replacement with KRaft.
2. **What is KRaft?**
   → Kafka's built-in metadata management using the Raft consensus algorithm, eliminating the need for Zookeeper.
3. **What is the Raft consensus algorithm?**
   → A protocol where a group of nodes elect a leader and commit changes only when a majority acknowledges — ensuring consistency even during failures.
4. **How many controllers should you have in production?**
   → 3 or 5 dedicated controllers forming the KRaft quorum.
5. **What Kafka version made KRaft production ready?**
   → Kafka 3.3. Zookeeper was fully removed in Kafka 4.

🔵 Topic 6: Kafka Producers

📌 Part 1: What is a Kafka Producer?

A Kafka Producer is any application that **publishes (writes) messages** to a Kafka topic.

[Your Java App] → [Kafka Producer API] → [Kafka Broker] → [Topic/Partition]

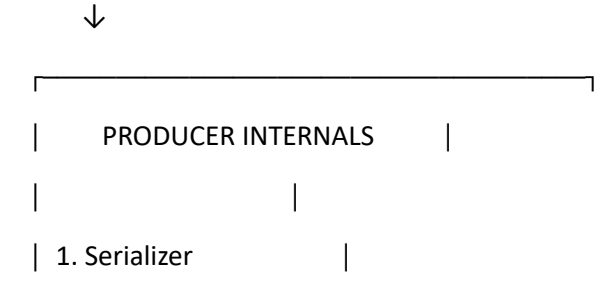
Real World Examples of Producers:

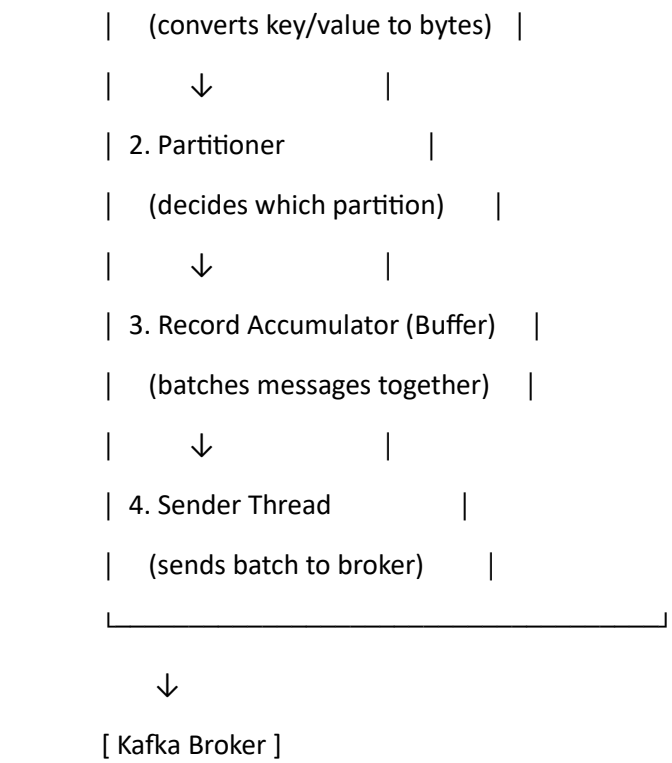
- Order service publishing order-placed events
- Payment service publishing payment-processed events
- IoT sensor sending temperature readings every second
- Application sending logs to Kafka for centralized logging

📌 Part 2: Producer Internals (How it works inside)

This is very important for interviews! Understanding what happens **inside the producer** before a message even reaches Kafka.

Your Code calls producer.send(record)





Let's understand each step:

Step 1: Serializer

- Kafka only understands **bytes**
- Serializer converts your Java objects → bytes before sending

```
java
// Common Serializers
props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
    "org.apache.kafka.common.serialization.StringSerializer");

props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
    "org.apache.kafka.common.serialization.StringSerializer");

// For JSON (using Spring Kafka)
props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
    "org.springframework.kafka.support.serializer.JsonSerializer");
```

Step 2: Partitioner

Decides **which partition** the message goes to:

If Key exists:

partition = murmur2_hash(key) % num_partitions

→ Same key always → same partition ✅

If Key is null:

→ Round Robin across partitions (Kafka 2.4+: Sticky Partitioner)

→ Batches messages to same partition until batch is full

→ Then moves to next partition

Step 3: Record Accumulator (Buffer)

- Messages are NOT sent immediately one by one
- They are **buffered** in memory and **batched together**
- This dramatically improves throughput

Record Accumulator (Memory Buffer):

Partition 0 batch: [msg1, msg2, msg3] → waiting to be sent

Partition 1 batch: [msg4, msg5] → waiting to be sent

Partition 2 batch: [msg6] → waiting to be sent

Two configs control when a batch is sent:

Config	Default	Meaning
batch.size	16384 (16KB)	Send batch when it reaches this size
linger.ms	0 ms	Wait this long before sending even if batch not full

linger.ms = 0 → Send immediately (low latency, less batching)

linger.ms = 5 → Wait 5ms to accumulate more messages (higher throughput)

Step 4: Sender Thread

- A **background thread** that picks batches from the accumulator
- Sends them to the appropriate broker/partition
- Handles retries if sending fails

📌 Part 3: ProducerRecord

Every message sent by a producer is a ProducerRecord:

java

```
ProducerRecord<String, String> record = new ProducerRecord<>(
    "order-events",    // topic
    "order-123",       // key (optional)
    "Order placed"     // value
);

// With partition specified explicitly
ProducerRecord<String, String> record = new ProducerRecord<>(
    "order-events",    // topic
    0,                 // partition (optional)
    "order-123",       // key
    "Order placed"     // value
);
```

📌 Part 4: Acknowledgment Modes (acks)

This is one of the **most important producer concepts** for interviews!

acks controls how many broker acknowledgments the producer requires before considering a message successfully sent.

---

acks = 0 (Fire and Forget)

Producer → sends message → does NOT wait for any acknowledgment

Broker 1 (Leader): receives message... maybe ✅ or maybe ❌

Producer: doesn't care, already moved on

Aspect	Value
Speed	⚡ Fastest

Aspect	Value
Durability	❌ Lowest — messages can be lost
Use case	Metrics, logs where some loss is acceptable

---

acks = 1 (Leader Acknowledgment) — Default

Producer → sends message → waits for Leader to acknowledge

Broker 1 (Leader): receives & stores message → sends ACK ✅

Producer: message confirmed ✅

But Followers may not have synced yet...

If Leader crashes before replication → message LOST 💥

Aspect	Value
Speed	⚡ Fast
Durability	Medium — leader acknowledged but not replicated
Use case	General purpose, moderate reliability

---

acks = all (or -1) — Strongest Guarantee

Producer → sends message → waits for Leader + ALL ISR replicas to acknowledge

Broker 1 (Leader): stores message ✅

Broker 2 (Follower): replicates ✅

Broker 3 (Follower): replicates ✅

All ISR acknowledged → Producer gets ACK ✅

Even if Leader crashes → Followers have the data → No loss ✅

Aspect	Value
Speed	Slower (waits for all ISR)

Aspect	Value
Durability	✅ Highest — no data loss
Use case	Financial transactions, critical events

---

acks Comparison Summary

acks=0: Producer —→ Broker (no ACK)

acks=1: Producer —→ Broker Leader —→ ACK

acks=all: Producer —→ Broker Leader

↓ replicates

Follower 1 —→ ACK

Follower 2 —→ ACK

↓

Leader —→ ACK to Producer

📌 **Part 5: Retries**

If sending fails (network issue, broker unavailable), the producer can **automatically retry**.

java

// Retry configuration

props.put(ProducerConfig.RETRIES_CONFIG, 3);

props.put(ProducerConfig.RETRY_BACKOFF_MS_CONFIG, 100); // wait 100ms between retries

Problem with Retries: Duplicate Messages

Producer sends message → Broker receives & stores ✅

Broker sends ACK → Network failure ❌ → Producer doesn't receive ACK

Producer retries → Sends same message again

Broker stores it AGAIN → Duplicate! 💥

Solution → **Idempotent Producer**

📌 **Part 6: Idempotent Producer**

An idempotent producer ensures that **even if a message is sent multiple times, it is stored only once**.

java

// Enable idempotence

props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);

How it works:

Producer gets a unique Producer ID (PID) from the broker

Each message gets a sequence number

Producer sends: PID=1, Seq=1, msg="order-placed"

Broker stores it ✅

Network failure → Producer retries:

Producer sends: PID=1, Seq=1, msg="order-placed" (same sequence!)

Broker sees: "I already stored PID=1, Seq=1" → IGNORES duplicate ✅

What idempotence automatically sets:

java

// When enable.idempotence=true, Kafka automatically sets:

acks = all // strongest durability

retries = MAX_INT // retry forever

max.in.flight.requests.per.connection = 5 // limits concurrent requests

---

📌 Part 7: Compression

Producers can compress messages before sending to reduce network bandwidth and storage.

java

props.put(ProducerConfig.COMPRESSION_TYPE_CONFIG, "snappy");

// Options: none, gzip, snappy, lz4, zstd

Algorithm	Speed	Compression Ratio	Use Case
none	N/A	None	Default
gzip	Slow	High	Storage saving priority
snappy	Fast	Medium	General purpose

Algorithm	Speed	Compression Ratio	Use Case
lz4	Very Fast	Medium	Low latency priority
zstd	Fast	High	Best overall (Kafka 2.1+)

Without compression:

Producer → [100KB batch] → Broker

With snappy compression:

Producer → [40KB batch] → Broker → decompresses → stores original

Compression happens at **batch level** — more messages in a batch = better compression ratio.

📌 Part 8: Important Producer Configs Summary

java

```
Properties props = new Properties();

// Connection
props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");

// Serialization
props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, StringSerializer.class);

// Reliability
props.put(ProducerConfig.ACKS_CONFIG, "all");
props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);
props.put(ProducerConfig.RETRIES_CONFIG, 3);

// Performance
props.put(ProducerConfig.BATCH_SIZE_CONFIG, 32768);       // 32KB batch
props.put(ProducerConfig.LINGER_MS_CONFIG, 5);            // wait 5ms
props.put(ProducerConfig.BUFFER_MEMORY_CONFIG, 33554432); // 32MB buffer
```

```
        props.put(ProducerConfig.COMPRESSION_TYPE_CONFIG, "snappy");
```

👉 *Kafka producer batches messages per partition in memory and sends them either when the batch is full or when the linger time expires, improving throughput by reducing network calls.*

📌 **Part 9: Producer in Spring Boot**

```
@Service
public class OrderProducer {

    @Autowired
    private KafkaTemplate<String, OrderEvent> kafkaTemplate;

    public void sendOrder(OrderEvent event) {
        // Simple send
        kafkaTemplate.send("order-events", event.getOrderId(), event);
    }

    public void sendWithCallback(OrderEvent event) {
        // Send with callback to handle success/failure
        kafkaTemplate.send("order-events", event.getOrderId(), event)
            .whenComplete((result, ex) -> {
                if (ex == null) {
                    System.out.println("Sent successfully to partition: "
                        + result.getRecordMetadata().partition()
                        + " offset: "
                        + result.getRecordMetadata().offset());
                } else {
                    System.err.println("Failed to send: " + ex.getMessage());
                }
            });
    }
```

}

yaml

```yaml
# application.yml
spring:
  kafka:
    producer:
      bootstrap-servers: localhost:9092
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
      acks: all
      retries: 3
      properties:
        enable.idempotence: true
        linger.ms: 5
        compression.type: snappy
```

🔑 **Key Takeaways**

Concept	One-line Summary
Serializer	Converts Java objects to bytes
Partitioner	Decides which partition a message goes to
Record Accumulator	Buffers messages into batches
batch.size	Max size of a batch before sending
linger.ms	Wait time to accumulate messages
acks=0	No acknowledgment — fastest, least reliable
acks=1	Leader acknowledges — moderate reliability
acks=all	All ISR acknowledge — slowest, most reliable
Idempotent Producer	Prevents duplicate messages on retry
Compression	Reduces batch size for better throughput

🎯 Common Interview Questions

1. **What is the difference between acks=1 and acks=all?**
   → acks=1 waits only for leader, acks=all waits for all ISR replicas. acks=all is safer but slower.
2. **How does idempotent producer work?**
   → Each message gets a unique Producer ID + sequence number. Broker deduplicates based on these.
3. **What is the purpose of linger.ms?**
   → Makes the producer wait before sending, allowing more messages to accumulate into a batch — improving throughput at the cost of slight latency.
4. **What happens if the buffer is full?**
   → Producer blocks for max.block.ms (default 60s), then throws BufferExhaustedException.
5. **When would you use acks=0?**
   → For use cases where some data loss is acceptable — metrics collection, clickstream data, application logs.

🔵 Topic 7: Kafka Consumers

📌 Part 1: What is a Kafka Consumer?

A Kafka Consumer is any application that **reads (subscribes to) messages** from a Kafka topic.

[Kafka Broker] → [Topic/Partition] → [Kafka Consumer API] → [Your Java App]

Real World Examples of Consumers:

- Notification service reading order-placed events to send SMS
- Analytics service reading click-events for real-time dashboards
- Fraud detection service reading payment-events to flag suspicious activity

---

📌 Part 2: Consumer Groups

This is the **most important concept** in Kafka consumers!

What is a Consumer Group?

A Consumer Group is a **set of consumers working together** to consume a topic, identified by a group.id.

java

props.put(ConsumerConfig.GROUP_ID_CONFIG, "notification-service-group");

Key Rule

Each partition is consumed by only ONE consumer within a group
But the SAME partition CAN be consumed by consumers in DIFFERENT groups

---

Example: Single Consumer Group

Topic: order-events (3 partitions)

Consumer Group: "notification-group"

├── Consumer 1 → reads Partition 0

├── Consumer 2 → reads Partition 1

└── Consumer 3 → reads Partition 2

Each message is processed exactly ONCE by this group ✅

Example: Multiple Consumer Groups (Pub-Sub style)

Topic: order-events (3 partitions)

Consumer Group: "notification-group"

├── Consumer 1 → reads Partition 0, 1, 2 (all of them)

Consumer Group: "analytics-group"

├── Consumer 1 → reads Partition 0, 1, 2 (all of them)

Both groups receive ALL messages independently! ✅

This is how Kafka supports Pub-Sub — each group gets a full copy

┌──→ Notification Group (gets all messages)

Topic: order-events ──┤

└──→ Analytics Group (gets all messages)

What happens if Consumers > Partitions?

Topic: order-events (3 partitions)

Consumer Group: "notification-group" with 5 consumers

Consumer 1 → Partition 0

Consumer 2 → Partition 1

Consumer 3 → Partition 2

Consumer 4 → IDLE ❌ (no partition to read)

Consumer 5 → IDLE ❌ (no partition to read)

⚠️ **Max useful consumers in a group = number of partitions**
Extra consumers sit idle — this is why partition count matters for scaling!

What happens if Partitions > Consumers?

Topic: order-events (6 partitions)

Consumer Group: "notification-group" with 3 consumers

Consumer 1 → Partition 0, Partition 1

Consumer 2 → Partition 2, Partition 3

Consumer 3 → Partition 4, Partition 5

A single consumer can read from **multiple partitions** — just not the reverse.

📌 **Part 3: Partition Rebalancing**

What is Rebalancing?

Rebalancing is the process of **reassigning partitions** to consumers within a group — triggered when:

- A new consumer **joins** the group
- An existing consumer **leaves/crashes**
- New partitions are **added** to the topic

Before:

Consumer 1 → Partition 0, 1

Consumer 2 → Partition 2, 3

Consumer 2 crashes 💥

Rebalancing triggered!

After:

Consumer 1 → Partition 0, 1, 2, 3 (took over all partitions)

Group Coordinator

Each consumer group has a **Group Coordinator** (a designated broker) that manages rebalancing.

1. Consumer sends heartbeat to Group Coordinator every few seconds

2. If heartbeat stops → Coordinator marks consumer as dead

3. Coordinator triggers rebalance

4. Coordinator picks one consumer as "Group Leader"

5. Group Leader decides new partition assignment

6. Coordinator broadcasts new assignment to all consumers

Types of Rebalancing Strategies

1. Eager Rebalancing (Old, default earlier)

ALL consumers STOP processing

↓

ALL partitions are revoked from everyone

↓

Partitions are reassigned from scratch

↓

Consumers resume

Problem: "Stop-the-world" — entire group pauses, even unaffected consumers ❌

2. Cooperative (Incremental) Rebalancing (Modern, recommended)

Only the AFFECTED partitions are reassigned

↓

Other consumers KEEP processing their existing partitions

↓

Minimal disruption ✅

java

```
// Enable cooperative rebalancing
props.put(ConsumerConfig.PARTITION_ASSIGNMENT_STRATEGY_CONFIG,
    "org.apache.kafka.clients.consumer.CooperativeStickyAssignor");
```

Partition Assignment Strategies

Strategy	Behavior
Range (default)	Assigns consecutive partitions per topic to each consumer
RoundRobin	Distributes partitions evenly across all consumers
Sticky	Tries to keep existing assignment, minimizes movement during rebalance
CooperativeSticky	Sticky + incremental rebalancing (best of both worlds) ⭐ recommended

java

```
props.put(ConsumerConfig.PARTITION_ASSIGNMENT_STRATEGY_CONFIG,
    "org.apache.kafka.clients.consumer.CooperativeStickyAssignor");
```

📌 Part 4: The Poll Loop

How Consumers Actually Read Messages

Kafka consumers don't get messages "pushed" to them — they **pull (poll)** messages in a loop.

java

```
KafkaConsumer<String, String> consumer = new KafkaConsumer<>(props);
consumer.subscribe(Collections.singletonList("order-events"));

while (true) {
    ConsumerRecords<String, String> records = consumer.poll(Duration.ofMillis(100));

    for (ConsumerRecord<String, String> record : records) {
        System.out.println("Partition: " + record.partition()
            + " Offset: " + record.offset()
            + " Value: " + record.value());
```

```
        // process the message
    }
}
```

What happens inside poll()?

1. Sends heartbeat to Group Coordinator (proves consumer is alive)
2. Checks if rebalancing is needed
3. Fetches new messages from assigned partitions
4. Returns records to your application code
5. Your code processes them
6. Loop continues...

⚠️ If your processing logic takes **too long**, the consumer may be considered "dead" by the coordinator (no heartbeat) → triggers unwanted rebalancing!

Important Poll Configs

java

```
props.put(ConsumerConfig.MAX_POLL_RECORDS_CONFIG, 500);       // max records per poll
props.put(ConsumerConfig.MAX_POLL_INTERVAL_MS_CONFIG, 300000); // max time between polls (5 min)
props.put(ConsumerConfig.SESSION_TIMEOUT_MS_CONFIG, 10000);    // heartbeat timeout (10 sec)
props.put(ConsumerConfig.HEARTBEAT_INTERVAL_MS_CONFIG, 3000);  // heartbeat frequency (3 sec)
```

max.poll.interval.ms → If processing takes longer than this between polls,

consumer is kicked out of the group (rebalance triggered)

session.timeout.ms → If no heartbeat received within this time,

consumer is considered dead

📌 **Part 5: Offset Management (Deep Dive)**

We touched on this in Topic 3, but let's go deeper since it's a hot interview area.

Auto Commit (Recap)

java

```
props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, true);
props.put(ConsumerConfig.AUTO_COMMIT_INTERVAL_MS_CONFIG, 5000);
```

Timeline:

T=0s: poll() returns msg1, msg2, msg3

T=0s: processing msg1... msg2... msg3...

T=5s: Auto-commit fires → commits offset (latest read position)

Risk: If crash happens AFTER processing but BEFORE auto-commit interval

→ On restart, messages are RE-processed (duplicate processing)

Manual Commit — Sync

java

```
props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, false);

while (true) {
   ConsumerRecords<String, String> records = consumer.poll(Duration.ofMillis(100));

   for (ConsumerRecord<String, String> record : records) {
      process(record);  // your business logic
   }

   consumer.commitSync(); // blocks until broker confirms commit
}
```

- **Blocking** — waits for broker acknowledgment
- Safer but slower (adds latency to your loop)

Manual Commit — Async

java

```
consumer.commitAsync((offsets, exception) -> {
   if (exception != null) {
      System.err.println("Commit failed: " + exception.getMessage());
```

```
    }
});
```

- **Non-blocking** — doesn't wait for confirmation
- Faster but risk: if it fails, you might not know immediately (no automatic retry, since retrying async commits out of order can cause issues)

Best Practice: Commit Specific Offsets (Fine-grained control)

java

```
Map<TopicPartition, OffsetAndMetadata> offsetsToCommit = new HashMap<>();


for (ConsumerRecord<String, String> record : records) {
    process(record);


    offsetsToCommit.put(
        new TopicPartition(record.topic(), record.partition()),
        new OffsetAndMetadata(record.offset() + 1)
    );
}


consumer.commitSync(offsetsToCommit);
```

This gives precise control — useful when processing involves multiple partitions and you want exact offset tracking.

📌 **Part 6: Delivery Semantics (Very Popular Interview Topic!)**

At-Most-Once

1. Consumer commits offset FIRST

2. Consumer THEN processes message

If crash happens after commit but before processing:

→ Message is LOST (never processed) ❌

→ But never duplicated

java

consumer.commitSync(); // commit BEFORE processing

process(record);

At-Least-Once ⭐ (Most commonly used)

1. Consumer processes message FIRST

2. Consumer THEN commits offset

If crash happens after processing but before commit:

→ On restart, message is processed AGAIN (duplicate) ⚠️

→ But never lost ✅

java

process(record);

consumer.commitSync(); // commit AFTER processing

This requires your processing logic to be **idempotent** (safe to process the same message multiple times) — e.g., using a unique message ID to detect duplicates in your database.

Exactly-Once

Message is processed EXACTLY once — no loss, no duplication

Achieved via:

- Kafka Transactions (producer + consumer combined)

- Idempotent producer + transactional consumer

- Typically used in Kafka Streams or "consume-transform-produce" pipelines

We'll cover this in depth in the **Reliability & Guarantees** topic.

Semantics Comparison

Semantic	Message Loss?	Duplicate Processing?	Use Case
At-most-once	Possible ❌	No	Metrics, logs (some loss OK)
At-least-once	No ✅	Possible ⚠️	Most business applications
Exactly-once	No ✅	No ✅	Financial transactions, critical data

📌 Part 7: Consumer Lag

What is Consumer Lag?

The **difference** between the latest message offset in a partition and the offset the consumer has processed.

Partition 0:

Latest offset in broker: 1000

Consumer's committed offset: 950

Consumer Lag = 1000 - 950 = 50 messages behind

Why does Lag matter?

- High lag = consumer is **falling behind** = potential data processing delays
- Used to decide if you need **more consumers/partitions**

bash

```
# Check consumer lag via CLI
kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --describe --group notification-group
```

Output:

```
TOPIC        PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
order-events  0          950             1000            50
order-events  1          800             800             0
order-events  2          700             750             50
```

Monitored via tools like **Grafana + Prometheus** or **Confluent Control Center**.

📌 Part 8: Consumer in Spring Boot

java

```
@Service
public class OrderConsumer {

    @KafkaListener(topics = "order-events", groupId = "notification-group")
    public void consume(ConsumerRecord<String, String> record) {
```

```
        System.out.println("Received: " + record.value()
            + " from partition: " + record.partition()
            + " at offset: " + record.offset());

        // process message
    }
}
```

yaml

```
# application.yml
spring:
  kafka:
    consumer:
      bootstrap-servers: localhost:9092
      group-id: notification-group
      auto-offset-reset: earliest
      enable-auto-commit: false
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
    listener:
      ack-mode: MANUAL_IMMEDIATE   # manual offset commit
```

Manual Acknowledgment in Spring Kafka

java

```
@KafkaListener(topics = "order-events", groupId = "notification-group")
public void consume(ConsumerRecord<String, String> record, Acknowledgment ack) {
    try {
        process(record);
        ack.acknowledge(); // manually commit offset after successful processing
    } catch (Exception e) {
        // don't acknowledge - message will be redelivered
        log.error("Processing failed", e);
    }
```

}

🔑 Key Takeaways

Concept	One-line Summary
Consumer Group	Set of consumers sharing work via group.id
Partition Rule	One partition → one consumer per group (max)
Rebalancing	Reassigning partitions when group membership changes
Cooperative Rebalancing	Modern, minimal-disruption rebalancing strategy
Poll Loop	Consumers pull messages; must poll within max.poll.interval.ms
At-most-once	Commit before processing — risk of message loss
At-least-once	Commit after processing — risk of duplicates (most common)
Exactly-once	No loss, no duplicates — needs transactions
Consumer Lag	How far behind a consumer is from the latest message

---

🎯 Common Interview Questions

1. **What happens if you have more consumers than partitions?**
   → Extra consumers sit idle. Max active consumers = number of partitions.
2. **What triggers a rebalance?**
   → Consumer joining/leaving the group, consumer crash, or new partitions added to topic.
3. **What's the difference between at-least-once and at-most-once?**
   → At-least-once commits after processing (may duplicate, never loses data). At-most-once commits before processing (may lose data, never duplicates).
4. **How do you make your consumer idempotent?**
   → Use a unique message ID/key, check if already processed (e.g., via DB unique constraint) before applying business logic again.
5. **What happens if processing takes longer than max.poll.interval.ms?**
   → Consumer is considered dead, removed from group, triggers rebalance — its partitions are reassigned to other consumers.
6. **Can two consumers in the same group read the same partition?**
   → No. Only one consumer per partition within a group at any time.

🔵 Topic 8: Kafka with Java (Spring Boot)

📌 Part 1: Spring Kafka Overview

Spring Kafka is a wrapper around the native Kafka Java client that makes it much easier to integrate Kafka into Spring Boot applications.

Native Kafka Client:

- Manual poll loop

- Manual offset management

- Manual error handling

- More boilerplate

Spring Kafka:

- @KafkaListener annotation does the poll loop for you

- KafkaTemplate simplifies producing

- Built-in retry, error handling, DLT support

- Less boilerplate ✅

📌 Part 2: Complete Producer Configuration

java

```
@Configuration
public class KafkaProducerConfig {

    @Value("${spring.kafka.bootstrap-servers}")
    private String bootstrapServers;

    @Bean
    public ProducerFactory<String, Object> producerFactory() {
        Map<String, Object> configProps = new HashMap<>();
        configProps.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapServers);
        configProps.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
```

```
        configProps.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, JsonSerializer.class);
        configProps.put(ProducerConfig.ACKS_CONFIG, "all");
        configProps.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);
        configProps.put(ProducerConfig.RETRIES_CONFIG, 3);
        return new DefaultKafkaProducerFactory<>(configProps);
    }

    @Bean
    public KafkaTemplate<String, Object> kafkaTemplate() {
        return new KafkaTemplate<>(producerFactory());
    }
}
```

📌 **Part 3: Complete Consumer Configuration**

```
java
@Configuration
@EnableKafka
public class KafkaConsumerConfig {

    @Value("${spring.kafka.bootstrap-servers}")
    private String bootstrapServers;

    @Bean
    public ConsumerFactory<String, Object> consumerFactory() {
        Map<String, Object> configProps = new HashMap<>();
        configProps.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapServers);
        configProps.put(ConsumerConfig.GROUP_ID_CONFIG, "order-group");
        configProps.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class);
        configProps.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, JsonDeserializer.class);
        configProps.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");
        configProps.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, false);
```

```
        configProps.put(JsonDeserializer.TRUSTED_PACKAGES, "*");
        return new DefaultKafkaConsumerFactory<>(configProps);
    }

    @Bean
    public ConcurrentKafkaListenerContainerFactory<String, Object> kafkaListenerContainerFactory() {
        ConcurrentKafkaListenerContainerFactory<String, Object> factory =
            new ConcurrentKafkaListenerContainerFactory<>();
        factory.setConsumerFactory(consumerFactory());
        factory.setConcurrency(3); // number of consumer threads (like 3 consumers in the group)

factory.getContainerProperties().setAckMode(ContainerProperties.AckMode.MANUAL_IMMEDIATE);
        return factory;
    }
}
```

💡 setConcurrency(3) spins up 3 consumer threads within your app instance — like having 3 consumers in the group, each handling different partitions.

📌 **Part 4: Error Handling in Spring Kafka**

The Problem

Consumer reads a "poison pill" message (malformed/bad data)

↓

Processing throws exception

↓

Without proper handling:

→ Same message gets retried infinitely

→ Consumer gets STUCK on this message forever

→ All messages behind it in the partition are blocked too! 💥

Solution: DefaultErrorHandler with Retry

java

```
@Bean
public DefaultErrorHandler errorHandler() {
    // Retry 3 times, with 1 second between attempts
    FixedBackOff backOff = new FixedBackOff(1000L, 3);

    DefaultErrorHandler errorHandler = new DefaultErrorHandler(backOff);

    // After retries exhausted, log and skip (or send to DLT)
    errorHandler.setRetryListeners((record, ex, deliveryAttempt) -> {
        log.warn("Retry attempt {} failed for record: {}", deliveryAttempt, record.value());
    });

    return errorHandler;
}
```

```
java
@Bean
public ConcurrentKafkaListenerContainerFactory<String, Object> kafkaListenerContainerFactory(
        DefaultErrorHandler errorHandler) {
    ConcurrentKafkaListenerContainerFactory<String, Object> factory =
        new ConcurrentKafkaListenerContainerFactory<>();
    factory.setConsumerFactory(consumerFactory());
    factory.setCommonErrorHandler(errorHandler);
    return factory;
}
```

📌 Part 5: Dead Letter Topic (DLT)

What is a DLT?

A **Dead Letter Topic** is a separate topic where messages that **repeatedly fail processing** are sent — instead of blocking the main topic forever.

order-events topic:

msg1 ✅ processed

msg2 ❌ fails → retry 3 times → still fails → sent to order-events.DLT

msg3 ✅ processed (not blocked by msg2!)

msg4 ✅ processed

Implementation with DeadLetterPublishingRecoverer

java

```
@Bean
public DefaultErrorHandler errorHandler(KafkaTemplate<String, Object> kafkaTemplate) {

    // Recoverer: where to send failed messages after retries exhausted
    DeadLetterPublishingRecoverer recoverer = new DeadLetterPublishingRecoverer(
        kafkaTemplate,
        (record, exception) -> new TopicPartition(record.topic() + ".DLT", record.partition())
    );

    FixedBackOff backOff = new FixedBackOff(1000L, 3); // retry 3 times, 1s apart

    return new DefaultErrorHandler(recoverer, backOff);
}
```

Flow:

1. Message fails processing
2. Retried 3 times (with 1 second gap)
3. Still failing → DeadLetterPublishingRecoverer kicks in
4. Message published to "order-events.DLT" topic
5. Original consumer moves on to next message ✅
6. A separate process/team can investigate DLT messages later

---

Consuming from DLT (for monitoring/alerting)

java

```
@KafkaListener(topics = "order-events.DLT", groupId = "dlt-monitor-group")
public void handleDltMessages(ConsumerRecord<String, Object> record) {
```

```
    log.error("Message in DLT: {} | Partition: {} | Offset: {}",
        record.value(), record.partition(), record.offset());

    // Send alert to team, store in DB for manual review, etc.
    alertService.notifyTeam(record);
}
```

📌 Part 6: Retry Mechanisms — Deep Dive

Fixed Backoff

```
java
FixedBackOff backOff = new FixedBackOff(1000L, 3);
// Wait 1000ms between each retry, max 3 retries
```

Exponential Backoff (More production-friendly)

```
java
ExponentialBackOff backOff = new ExponentialBackOff();
backOff.setInitialInterval(1000L);   // first retry after 1s
backOff.setMultiplier(2.0);          // double the wait each time
backOff.setMaxInterval(10000L);      // cap at 10s
backOff.setMaxElapsedTime(30000L);   // give up after 30s total

// Retry timeline: 1s → 2s → 4s → 8s → 10s (capped) → give up
```

Why Exponential Backoff?

If a downstream service is overwhelmed:

Fixed backoff: hammers it every 1s → makes it worse 😰

Exponential backoff: backs off progressively → gives system time to recover ✅

📌 Part 7: Retryable vs Non-Retryable Exceptions

Not all errors should be retried! Some errors will **always** fail no matter how many times you retry (e.g., deserialization errors, null pointer due to bad data structure).

```
java
@Bean
```

```
public DefaultErrorHandler errorHandler(KafkaTemplate<String, Object> kafkaTemplate) {

    DeadLetterPublishingRecoverer recoverer = new DeadLetterPublishingRecoverer(kafkaTemplate);

    DefaultErrorHandler errorHandler = new DefaultErrorHandler(recoverer, new FixedBackOff(1000L,
3));

    // Don't retry these — send straight to DLT
    errorHandler.addNotRetryableExceptions(
        IllegalArgumentException.class,
        NullPointerException.class,
        DeserializationException.class
    );

    // Only retry these
    errorHandler.addRetryableExceptions(
        TimeoutException.class,
        ConnectException.class
    );

    return errorHandler;
}
```

Retryable errors (temporary, may succeed later):

- Network timeout
- Database connection failure
- Downstream service temporarily down

Non-retryable errors (permanent, will never succeed):

- Malformed JSON / Deserialization failure
- Null pointer due to missing required field
- Business validation failure (e.g., invalid order amount)

📌 Part 8: Serialization & Deserialization (Deep Dive)

String Serialization

java

configProps.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, StringSerializer.class);

Simple, but you lose type safety — everything is just a String.

JSON Serialization (Most Common in Spring Boot)

java

// Producer side

configProps.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, JsonSerializer.class);

java

// Consumer side

configProps.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, JsonDeserializer.class);

configProps.put(JsonDeserializer.TRUSTED_PACKAGES, "com.example.events"); // security: whitelist packages

configProps.put(JsonDeserializer.VALUE_DEFAULT_TYPE, OrderEvent.class);

java

public class OrderEvent {

  private String orderId;

  private String status;

  private BigDecimal amount;

  // getters, setters, constructors

}

// Producer sends Java object directly

kafkaTemplate.send("order-events", orderEvent); // auto-converted to JSON

// Consumer receives it as Java object directly

@KafkaListener(topics = "order-events")

public void consume(OrderEvent event) { // auto-converted from JSON

```
    System.out.println(event.getOrderId());
}
```

Avro Serialization (More advanced, used with Schema Registry)

We'll cover this in detail in the **Schema Registry & Avro** topic — it provides schema validation and evolution, unlike JSON.

📌 Part 9: Kafka Transactions in Spring

The Use Case: "Consume-Transform-Produce" Pattern

Scenario: Read from "raw-orders" topic, transform, write to "processed-orders" topic

+ also update database

Problem: What if you write to Kafka but DB update fails (or vice versa)?

→ Data inconsistency!

✅ Solutions (Real-world patterns)

🔷 1. ❌ Naive approach (don't use)	🔷 2. ✅ Kafka Transactions (Partial Solution)
Process → Write Kafka → Update DB 👉 No guarantees → inconsistent system	Kafka transactions allow you to atomically do: 1. Consume messages 2. Process them 3. Produce new messages to another topic 4. Commit consumer offsets ✅ All of the above happen **as a single atomic unit** 🔴 **The Limitation: Database is NOT Included** 👉 Kafka transactions = **only within Kafka boundary** They guarantee: Kafka Topic A → Kafka Topic B → Offset commit But NOT: Kafka ↔ Database 💡 **Why This Happens** Because Kafka and your DB: • Are **two different systems** • Have **separate transaction managers** • No shared

🔷 3. ✅ Idempotent Consumer + Retry (Very common)

🚨 **Without Idempotency (Problem)**

Message: order_id = 101

1st processing → DB updated ✅

Crash happens before offset commit ❌

Kafka retries…

2nd processing → DB updated again ❌ (duplicate!)

💥 Result:

- Duplicate rows
- Wrong balance
- Corrupted data

✅ **With Idempotent Consumer (Solution)**

We make DB operation **safe to repeat**

👉 So even if message is processed 2, 3, 10 times:

- Final DB state remains correct

✅ **1. Use a Unique Key (Very Important)**

Example:

order_id = UNIQUE

Now DB enforces:

No duplicate entries for same order

✅ **2. Use UPSERT Instead of INSERT**

Instead of:

```
INSERT INTO orders (order_id, amount)
VALUES (101, 500);
```

Use:

```
INSERT INTO orders (order_id, amount)
VALUES (101, 500)
ON CONFLICT (order_id)
DO UPDATE SET amount = EXCLUDED.amount;
```

👉 This means:

- If record exists → update
- If not → insert

✓ Safe for retries

🔁 **Flow with Retry (Correct Behavior)**

1. Kafka message received
2. DB UPSERT executed
3. Kafka produce / offset commit fails ❌

🔷 4. ✅ Outbox Pattern (Best Practice ⭐)

Instead of doing:

DB + Kafka directly

We do:

👉 **Step 1: Save EVERYTHING in DB first**

We create 2 tables:

🗄️ **orders table**

order_id = 101

status = CREATED

📦 **outbox table (NEW IDEA)**

event = "ORDER_CREATED"

order_id = 101

status = PENDING

🧠 **Key idea**

👉 "If DB transaction succeeds, nothing is lost"

So now:

DB transaction:

- Save order
- Save event in outbox

→ COMMIT

✓ Both saved together OR none saved

🔁 **Step 2: Separate worker sends to Kafka**

Now another process runs:

Outbox table → Kafka

It does:

1. Read PENDING events
2. Send to Kafka
3. Mark as SENT

🔥 **Why this is powerful**

Because now:

✓ **No data loss**

Even if Kafka fails → event is still in DB

✓ **No inconsistency**

DB and event are ALWAYS together

✓ **Safe retry**

Even if processor crashes → it will retry later

4. Message retried Retry: 5. Same message processed again 6. UPSERT runs again → no duplicate effect ✅ 💡 Final DB state is still correct	

🔥 **1. Core Kafka Concepts (Expected Strong Fundamentals)**

1. What is Kafka and why is it used instead of traditional messaging systems?
2. What are brokers, topics, partitions, and offsets?
3. What is the role of a partition in Kafka?
4. How does Kafka achieve scalability?
5. What is a consumer group?
6. How does Kafka distribute messages across consumers in a group?
7. What happens if a new consumer joins a consumer group?
8. What is the difference between key-based and round-robin partitioning?

⚡ **2. Delivery Semantics & Reliability (VERY IMPORTANT)**

9. What is at-most-once, at-least-once, and exactly-once delivery?
10. Which delivery guarantee does Kafka provide by default?
11. How do duplicates happen in Kafka?
12. How do you handle duplicate messages in production systems?
13. Can Kafka guarantee no message loss? Explain how.

🔄 **3. Offset Management & Consumer Behavior**

14. What is an offset in Kafka?
15. What is the difference between auto commit and manual commit?
16. What happens if a consumer crashes before committing offset?
17. What is consumer lag?
18. How do you monitor and reduce consumer lag?
19. What happens during consumer rebalance?
20. How does Kafka ensure message ordering?

🧠 **4. Producer Side (Important for Real Systems)**

21. What is a Kafka producer?
22. What is idempotent producer?
23. What is acks (0, 1, all)?
24. What happens if acks=0 vs acks=all?
25. What is batching in Kafka producer?
26. How does compression improve Kafka performance?

💥 **5. Failure Handling (VERY HIGH PRIORITY)**

27. What is a retry mechanism in Kafka?
28. What is exponential backoff?
29. What is a Dead Letter Queue (DLQ)?

30. What are poison pill messages?
31. How do you handle continuously failing messages?
32. What happens when a broker goes down?

🏗️ 6. Real System Design Scenarios (MOST IMPORTANT ⭐)

33. Design an order processing system using Kafka.
34. How would you design a payment system using Kafka?
35. How do you ensure no duplicate orders in Kafka?
36. What happens if DB write succeeds but Kafka publish fails?
37. How do you handle consistency between Kafka and database?
38. What is the Outbox Pattern? Why is it needed?
39. What is CDC (Change Data Capture)? Where is it used?

🚀 7. Kafka Scalability & Performance

40. How do partitions help in scaling Kafka?
41. How do you decide number of partitions?
42. What is throughput vs latency tradeoff?
43. How do you tune Kafka for high performance?
44. What is message batching?
45. What is consumer parallelism?

🔥 8. Kafka Internals & Fault Tolerance

46. What is replication factor?
47. What is ISR (In-Sync Replicas)?
48. What happens when leader broker fails?
49. What is min.insync.replicas?
50. How does Kafka ensure durability?
51. What happens if all brokers are down?

🔄 9. Advanced Concepts (4–6 yrs expected awareness)

52. What is Kafka Streams?
53. What is exactly-once processing in Kafka?
54. What is transactional producer?
55. Can Kafka transactions work with database transactions?
56. What is schema registry (Avro/Protobuf)?
57. What is event-driven architecture?

🧩 10. Spring Boot + Kafka (Very Common in Java Interviews)

58. How do you implement Kafka consumer in Spring Boot?
59. What is @KafkaListener?
60. How do you configure manual offset commit?
61. How do you implement retry in Spring Kafka?
62. How do you implement DLQ in Spring Kafka?
63. How do you handle deserialization errors?

💣 11. Tricky Real-World Questions (SENIOR LEVEL ⭐)

64. How do you handle duplicate message processing in production?

65. How do you recover from partial failures?
66. What happens if Kafka produces but DB update fails?
67. How do you guarantee exactly-once processing in a microservice system?
68. How do you handle large message payloads?
69. What are common production issues in Kafka systems?
70. How do you debug consumer lag issues?

🎯 **What interviewers REALLY expect at 4–6 years**
They don't want definitions.
They expect:
✓ System design thinking
✓ Failure handling
✓ DB + Kafka consistency understanding
✓ Real production patterns (Outbox, DLQ, Idempotency)
✓ Ability to explain trade-offs

How to setup KAFKA without Zookeeper and without installing it on local machine.

Steps
1. Download Kafka
Go to the Kafka downloads page and grab the latest binary release (not the source). It'll be a .tgz file, something like:
kafka_2.13-3.9.0.tgz
(Scala version doesn't matter much for you — 2.13 is fine.)

2. Extract it
bash
tar -xzf kafka_2.13-3.9.0.tgz
cd kafka_2.13-3.9.0
That's it — it's "installed." No system PATH changes needed if you just call scripts with their full/relative path.

Prerequisites check
- Java must be installed (JDK 11 or 17+). Check with:

java -version
If that fails, you'll need a JDK first — this is the one thing you can't avoid installing.
(Adoptium/Temurin zip builds work too if you want a no-installer JDK — let me know if you need that.)
3.1 — Open Command Prompt or PowerShell and go to your Kafka folder
D:
cd D:\kafka_2.13-3.9.0
3.2 — Set a storage directory on D drive (important)
By default, Kafka's config points to a temp folder on C drive for logs/data. Since you want everything on D, edit the config file first.
Open this file in Notepad:

D:\kafka_2.13-3.9.0\config\kraft\server.properties
Find this line:
log.dirs=/tmp/kraft-combined-logs
Replace it with a Windows path on D:
log.dirs=D:/kafka_2.13-3.9.0/data
Save the file. (Forward slashes are fine on Windows for Kafka configs.)

3.3 — Generate a Cluster ID
In your terminal (still inside the kafka folder):
bin\windows\kafka-storage.bat random-uuid
This prints a UUID like xtzWWN4bTjitpL3kfd9s5g. Copy it — you'll use it in the next step.

3.4 — Format the storage directory
bin\windows\kafka-storage.bat format -t xtzWWN4bTjitpL3kfd9s5g -c config\kraft\server.properties
(Replace the UUID with the one you actually generated.) This creates the D:\kafka_2.13-3.9.0\data folder and initializes it.

3.5 — Start Kafka
bin\windows\kafka-server-start.bat config\kraft\server.properties
Leave this terminal window open — this is your running Kafka broker. You'll see log output; look for a line indicating the Kafka server has started.
Kafka is now listening on localhost:9092.

Quick sanity test (optional, new terminal window)
Open a **second** Command Prompt, go to the same folder, and try:
Create a topic:
D:
cd D:\kafka_2.13-3.9.0
bin\windows\kafka-topics.bat --create --topic test-topic --bootstrap-server localhost:9092
List topics:
bin\windows\kafka-topics.bat --list --bootstrap-server localhost:9092
Produce a message:
bin\windows\kafka-console-producer.bat --topic test-topic --bootstrap-server localhost:9092
Type a message and hit Enter, then Ctrl+C to exit.
Consume it (new terminal):
bin\windows\kafka-console-consumer.bat --topic test-topic --bootstrap-server localhost:9092 --from-beginning
If you see your message printed, Kafka's working and your Spring Boot app can connect to localhost:9092.

To stop Kafka
Just go to the terminal running the broker and press Ctrl+C.
To clean up completely later
Close everything, then just delete the folder:
rmdir /s /q D:\kafka_2.13-3.9.0

A couple of things worth knowing before you start:

- Windows Kafka (especially older versions) has a known quirk with long file paths and antivirus locking log segment files — if you hit weird FileNotFoundException or "cannot delete" errors, it's usually AV interference or path-length issues, not a real problem. Using a short base path like D:\kafka instead of a deeply nested folder helps.
- Do you already have a JDK installed, or do you need help getting a portable/zip JDK too?

Connecting from Spring Boot
In application.properties:
properties
spring.kafka.bootstrap-servers=localhost:9092
spring.kafka.consumer.group-id=my-group
spring.kafka.consumer.auto-offset-reset=earliest
That's it — no Zookeeper, no installer, everything self-contained in D:\kafka_2.13-4.3.1.

One-time (already done, never again — unless you delete the data folder)

- Downloading/extracting Kafka
- Editing config\server.properties (log.dirs path)
- kafka-storage.bat format ... --standalone

Your topic (test-topic) and its data are saved on disk in D:\kafka\data (or wherever you pointed log.dirs). As long as you don't delete that folder, everything persists across restarts — you won't lose topics or messages.

Every time you want to use Kafka (e.g. new day, PC restart)
Just start the broker:
D:
cd D:\kafka_2.13-4.3.1
bin\windows\kafka-server-start.bat config\server.properties
That's genuinely it — one command. Leave that terminal open while you work, Ctrl+C to stop when done. Your topics and data are still there next time you start it.

Make it a one-click script (optional, saves typing)
Create a file start-kafka.bat inside your Kafka folder:
bat
@echo off
cd /d D:\kafka_2.13-4.3.1
bin\windows\kafka-server-start.bat config\server.properties
Double-click it whenever you want Kafka running — no need to remember the commands.

When you'd need to reformat (rare)
Only if you:

- Delete/corrupt the data folder
- Want to wipe everything and start fresh
- Move Kafka to a different path

For normal day-to-day dev work with Spring Boot, it's just: **start the broker, do your work, Ctrl+C when done.**